CRM

Mini Sistema de Gerenciamento de Clientes

Documentação técnica e guia de consultas SQL

PHP · MySQL/MariaDB · HTML · CSS · JavaScript

Novembro de 2026

📘 Documentação Técnica — Mini Sistema de Gerenciamento de Clientes

> Documentação completa do projeto: arquitetura, banco de dados, fluxo de cada página, front-end, segurança, problemas conhecidos e um guia de consultas SQL (do básico ao avançado).

Sumário

1. Visão geral
2. Tecnologias
3. Instalação e execução
4. Estrutura de arquivos
5. Arquitetura e fluxo de requisições
6. Banco de dados
7. Descrição de cada arquivo
8. Front-end (CSS e JavaScript)
9. Segurança
10. Problemas conhecidos e melhorias sugeridas
11. Guia de consultas SQL

---

1. Visão geral

O **Mini Sistema** é uma aplicação web CRUD (Create, Read, Update, Delete) para cadastro de clientes, escrita em PHP puro (sem framework), com banco MySQL/MariaDB acessado via **PDO**. Todo o sistema fica protegido por uma tela de **login** baseada em sessão PHP.

Funcionalidade	Página	Operação SQL principal
Login	`login.php`	`SELECT`
Logout	`logout.php`	—
Home	`index.php`	—
Cadastrar	`cadastro.php`	`INSERT`
Listar/Buscar	`clientes.php`	`SELECT`
Atualizar	`atualizar.php`	`SELECT` + `UPDATE`
Apagar	`apagar.php`	`SELECT` + `DELETE`

---

2. Tecnologias

Camada	Tecnologia
Servidor	Apache (XAMPP) ou servidor embutido do PHP
Back-end	PHP 7.4+ (`declare(strict_types=1)` em parte dos arquivos)
Banco de dados	MariaDB/MySQL (o XAMPP traz MariaDB)
Acesso a dados	PDO com *prepared statements*
Front-end	HTML5, CSS3 (variáveis CSS, layout responsivo), JavaScript puro

3. Instalação e execução

3.1 Pré-requisitos

- XAMPP instalado, com **Apache** e **MySQL** iniciados no painel de controle.
- Projeto dentro de `C:\xampp\htdocs\mini_sistema\mini_sistema`.

3.2 Criar o banco e as tabelas

No phpMyAdmin (`http://localhost/phpmyadmin`) ou no terminal do MySQL:

```
CREATE DATABASE IF NOT EXISTS mini_sistema
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE mini_sistema;
```

Depois, execute os dois scripts do projeto:

- usuarios.sql — cria a tabela `usuarios` (login) e o usuário administrador;
- clientes.sql — cria a tabela `clientes` (dados do CRUD).

Pelo terminal:

```
C:\xampp\mysql\bin\mysql.exe -u root mini_sistema < usuarios.sql
C:\xampp\mysql\bin\mysql.exe -u root mini_sistema < clientes.sql
```

3.3 Acessar

- Via Apache do XAMPP: `http://localhost/mini_sistema/mini_sistema/login.php`
- Ou via servidor embutido do PHP (na pasta do projeto), acessando depois `http://localhost:8000`:

```
php -S localhost:8000
```

Login padrão: `admin@sistema.com` / `admin123`

3.4 Configuração da conexão

Os dados de conexão ficam em conexao.php:

Variável	Valor padrão
`$host`	`127.0.0.1`
`$dbname`	`mini_sistema`
`$user`	`root`
`$password`	*(vazio)*

4. Estrutura de arquivos

```
mini_sistema/
├── conexao.php      # Cria o objeto $pdo (conexão com o banco)
├── sessao.php       # Inicia a sessão PHP, se ainda não iniciada
├── auth.php         # "Guarda" das páginas: exige usuário logado
├── login.php        # Formulário e processamento do login
├── logout.php       # Destrói a sessão e volta ao login
├── index.php        # Página inicial (Home)
├── cadastro.php     # Formulário de cadastro de cliente (INSERT)
├── clientes.php     # Listagem + busca de clientes (SELECT)
├── atualizar.php    # Edição de cliente (SELECT + UPDATE)
├── apagar.php       # Exclusão de cliente (SELECT + DELETE)
├── usuarios.sql     # Script da tabela `usuarios` + usuário admin
├── clientes.sql     # Script da tabela `clientes`
├── css/
│   └── styly.css    # Estilo de todas as páginas
├── js/
│   ├── cadastro.js  # Validação/máscara do formulário de cadastro
│   ├── atualizar.js # Validação/máscara do formulário de atualização
│   └── clientes.js  # Busca instantânea na tabela de clientes
├── imagens/
│   └── bem-vindo.jpeg
├── README.md        # Resumo do projeto
└── DOCUMENTACAO.md  # Este documento
```

5. Arquitetura e fluxo de requisições

O projeto segue o modelo **"uma página PHP = uma tela"**: cada arquivo processa o formulário (quando a requisição é `POST`) no topo e, em seguida, renderiza o HTML.

5.1 Cadeia de includes

```
página protegida (ex.: clientes.php)
   ├── require auth.php
   │       └── require sessao.php  → session_start()
   │       └── sem $_SESSION['usuario_id']? → redireciona p/ login.php?
redirect=clientes.php
   └── require conexao.php        → $pdo pronto para uso
```

5.2 Fluxo de login

```
Usuário acessa clientes.php sem estar logado
        │
        ▼
auth.php → Location: login.php?redirect=clientes.php
        │
        ▼
login.php (POST email + senha)
   ├── SELECT ID, NOME, EMAIL, SENHA FROM usuarios WHERE EMAIL = ?
   ├── password_verify(senha, hash)
   │     ├── OK   → session_regenerate_id(true)
   │     │          grava usuario_id / usuario_nome / usuario_email na sessão
   │     │          Location: clientes.php   (valor de redirect)
   │     └── Falha → "Email ou senha inválidos!"
```

O parâmetro `redirect` é validado com a regex `^[a-zA-Z0-9_\-]+\.php$`, impedindo redirecionamento para sites externos (*open redirect*).

5.3 Fluxo do CRUD

```
cadastro.php  ──POST──► INSERT  ──► mensagem de sucesso
clientes.php  ──GET───► SELECT ... ORDER BY ID DESC ──► tabela + busca JS
   ├── ✏️ Editar  → atualizar.php?id=N ──GET──► SELECT por ID
   │                                   ──POST─► UPDATE por ID
   └── 🗑️ Deletar → confirm() JS → apagar.php?id=N ──► SELECT + DELETE
                                                    └► Refresh 2s → clientes.php
```

5.4 Variáveis de sessão

Chave	Conteúdo
`$_SESSION['usuario_id']`	ID do usuário logado
`$_SESSION['usuario_nome']`	Nome exibido no cabeçalho
`$_SESSION['usuario_email']`	Email do usuário logado

6. Banco de dados

Banco: `mini_sistema` (charset `utf8mb4`).

6.1 Tabela `usuarios`

Usada para **autenticação**. Criada por usuarios.sql.

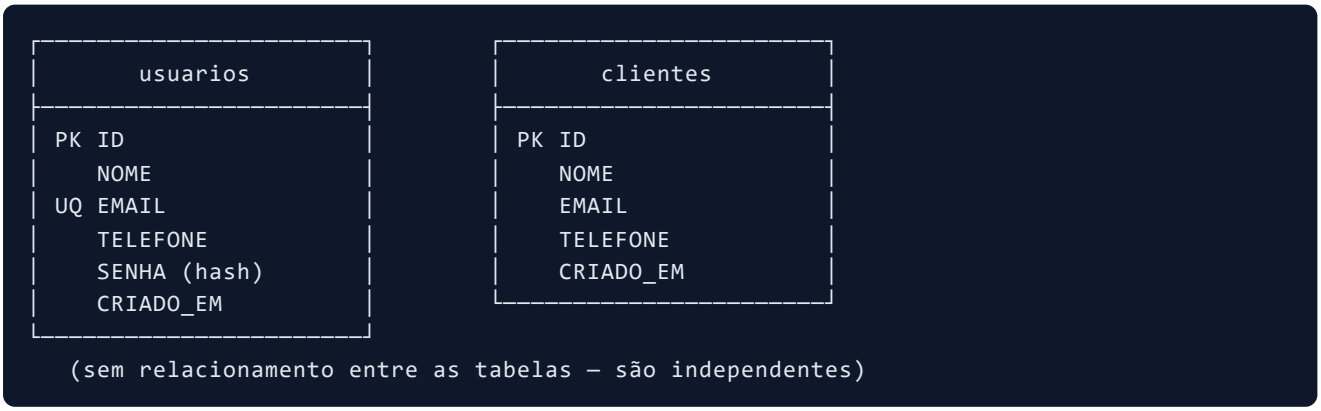
Coluna	Tipo	Restrições	Descrição
`ID`	`INT(11)`	PK, `AUTO_INCREMENT`	Identificador
`NOME`	`VARCHAR(100)`	`NOT NULL`	Nome do usuário
`EMAIL`	`VARCHAR(100)`	`NOT NULL`, `UNIQUE`	Login
`TELEFONE`	`VARCHAR(20)`	`NULL`	Telefone (opcional)
`SENHA`	`VARCHAR(255)`	`NOT NULL`	Hash bcrypt da senha
`CRIADO_EM`	`TIMESTAMP`	`DEFAULT CURRENT_TIMESTAMP`	Data de criação

6.2 Tabela `clientes`

Tabela dos **clientes** gerenciados pelo CRUD (cadastro, listagem, atualização e exclusão). Criada por clientes.sql. A coluna `CRIADO_EM` é preenchida automaticamente e é usada em várias consultas da seção 11:

```
CREATE TABLE IF NOT EXISTS clientes (
    ID        INT(11)      NOT NULL AUTO_INCREMENT,
    NOME      VARCHAR(100) NOT NULL,
    EMAIL     VARCHAR(100) NOT NULL,
    TELEFONE  VARCHAR(20)  NULL,
    CRIADO_EM TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (ID)
);
```

6.3 Diagrama



7. Descrição de cada arquivo

conexao.php

Cria a instância `$pdo` com três opções importantes:

- `ERRMODE_EXCEPTION` — erros SQL viram exceções (nada falha em silêncio);
- `FETCH_ASSOC` — resultados vêm como arrays associativos (`$linha['NOME']`);
- `EMULATE_PREPARES => false` — *prepared statements* reais no servidor.

Se a conexão falhar, responde HTTP 500 com uma mensagem amigável.

sessao.php

Chama `session_start()` somente se a sessão ainda não foi iniciada, evitando *warnings* quando incluído mais de uma vez.

auth.php

Incluído no topo de toda página protegida. Se não existir `$_SESSION['usuario_id']`, redireciona para `login.php?redirect=<página atual>` e encerra o script com `exit`.

login.php

- Se o usuário já estiver logado, vai direto para `index.php`.
- No `POST`, busca o usuário pelo email e confere a senha com `password_verify()`.
- Em caso de sucesso, regenera o ID da sessão e grava os dados do usuário.
- Mantém o email digitado no campo após um erro.

logout.php

Esvazia `$_SESSION`, expira o cookie de sessão, chama `session_destroy()` e volta ao login.

index.php

Página inicial estática: mensagem de boas-vindas, imagem, atalhos e cards de funcionalidades.

cadastro.php

Recebe `nome`, `email` e `telefone` por `POST`, valida campos obrigatórios (nome e email) e executa um `INSERT` com *prepared statement*.

clientes.php

Executa um `SELECT` ordenado por `ID DESC` (mais recentes primeiro) e monta a tabela. Cada linha recebe o atributo `data-search` (nome + email + telefone em minúsculas) usado pela busca em js/clientes.js. Mostra o total de registros no rodapé da tabela.

atualizar.php

- Lê `id` da URL (convertido com `(int)`), busca o registro e redireciona para a lista se não existir.
- No `POST`, valida e executa o `UPDATE`, atualizando também os valores exibidos no formulário.

apagar.php

Busca o registro pelo `id`, executa o `DELETE`, mostra uma mensagem e redireciona para a lista após 2 segundos (`header("Refresh: 2; url=clientes.php")`).

usuarios.sql

Cria a tabela `usuarios`, garante a coluna `TELEFONE` (sintaxe `ADD COLUMN IF NOT EXISTS`, própria do MariaDB) e insere o administrador padrão apenas se ele ainda não existir (`INSERT ... SELECT ... WHERE NOT EXISTS`).

clientes.sql

Cria a tabela `clientes` (se ainda não existir) e garante a coluna `CRIADO_EM` em tabelas criadas anteriormente sem ela.

---

8. Front-end (CSS e JavaScript)

8.1 css/styly.css

Folha de estilo única para todas as telas: cabeçalho com navegação, cards, formulários, tabela responsiva (`.table-responsive`), alertas (`.alert-success` / `.alert-error`), botões (`.btn-primary`, `.btn-secondary`, `.btn-warning`, `.btn-danger`, `.btn-sm`) e o layout da tela de login (`.login-body`, `.login-card`).

8.2 js/cadastro.js e js/atualizar.js

Comportamento praticamente idêntico:

Função	O que faz
validação no `blur`	Nome com menos de 3 caracteres ou email inválido → borda vermelha + dica
`isValidEmail()`	Regex simples `^[^\s@]+@[^\s@]+\.[^\s@]+$`
`formatarTelefone()`	Máscara automática: `(11) 9999-9999` ou `(11) 99999-9999` (máx. 11 dígitos)
`showTooltip()`	Cria/substitui a mensagem de erro abaixo do campo
validação no `submit`	Bloqueia o envio se nome ou email forem inválidos

`atualizar.js` também adiciona a classe `modificado` ao formulário quando algum campo muda.

8.3 js/clientes.js

Busca instantânea (sem recarregar a página): a cada tecla, esconde as linhas cujo `data-search` não contém o termo e atualiza o contador ("3 clientes encontrados").

---

9. Segurança

Medida	Onde
Prepared statements (anti SQL Injection)	Todas as consultas com parâmetros
Senhas com hash bcrypt	`password_hash` / `password_verify`
Regeneração do ID de sessão no login	`login.php` (`session_regenerate_id`)
Escape de saída (anti XSS)	`htmlspecialchars()` nos dados exibidos
Validação do `redirect` (anti open redirect)	`login.php` (regex)
Conversão do `id` para inteiro	`atualizar.php` , `apagar.php`
Confirmação antes de excluir	`confirm()` nos links de exclusão

---

10. Problemas conhecidos e melhorias sugeridas

10.1 ✅ Corrigido: mistura entre as tabelas `usuarios` e `clientes`

Antes, as páginas do CRUD usavam tabelas diferentes para a mesma operação (cadastro e listagem em `usuarios` , edição e exclusão em `clientes`). Isso fazia o cadastro falhar (`usuarios.SENHA` é `NOT NULL`), a lista mostrar os usuários do sistema em vez dos clientes, e editar/apagar não terem efeito visível.

Situação atual: cada tabela tem um único papel.

Arquivo	Leitura (`SELECT`)	Escrita
`login.php`	`FROM usuarios`	—
`cadastro.php`	—	`INSERT INTO clientes`
`clientes.php`	`FROM clientes`	—
`atualizar.php`	`FROM clientes`	`UPDATE clientes`
`apagar.php`	`FROM clientes`	`DELETE FROM clientes`

> Se você já cadastrou "clientes" na tabela `usuarios` antes da correção, eles continuam lá. Para copiá-los para `clientes` (ignorando o admin), use:

```
INSERT INTO clientes (NOME, EMAIL, TELEFONE)
SELECT NOME, EMAIL, TELEFONE FROM usuarios
WHERE EMAIL <> 'admin@sistema.com';
```

10.2 Outras melhorias

- Validar o formato do email também no servidor (`filter_var($email, FILTER_VALIDATE_EMAIL)`).
- Exclusão via `GET` pode ser disparada por um link externo (CSRF). O ideal é excluir via `POST` com um token CSRF.
- `cadastro.php` , `atualizar.php` e `apagar.php` imprimem `$mensagem` sem `htmlspecialchars()` (em `apagar.php` a mensagem contém o nome do cliente → risco de XSS armazenado).
- Tratar email duplicado (capturar `PDOException` código `23000`).
- Remover a dica de credenciais padrão da tela de login em produção.
- Ano do rodapé divergente (2024 × 2026) e cabeçalho diferente em `apagar.php` .
- Extrair o cabeçalho/rodapé repetidos para um `header.php` / `footer.php` .
- `cadastro.js` e `atualizar.js` duplicam funções; poderiam compartilhar um `validacao.js` .

---

11. Guia de consultas SQL

> Todas as consultas consideram o banco `mini_sistema` e as tabelas descritas na seção 6. Execute-as no phpMyAdmin (aba **SQL**) ou no cliente `mysql` . Antes de rodar, selecione o banco:

```
USE mini_sistema;
```

11.0 Dados de exemplo

Para testar as consultas, insira alguns clientes:

```
INSERT INTO clientes (NOME, EMAIL, TELEFONE, CRIADO_EM) VALUES
('Ana Souza',       'ana.souza@gmail.com',      '(11) 98765-4321', '2026-01-15 10:00:00'),
('Bruno Lima',      'bruno.lima@hotmail.com',   '(21) 99876-5432', '2026-02-03 14:30:00'),
('Carla Mendes',    'carla@empresa.com.br',     NULL,              '2026-02-20 09:15:00'),
('Diego Alves',     'diego.alves@gmail.com',    '(11) 3456-7890',  '2026-03-11 16:45:00'),
('Elisa Ferreira',  'elisa.f@yahoo.com',        '(31) 98888-7777', '2026-03-28 11:20:00'),
('Fábio Rocha',     'fabio.rocha@gmail.com',    '(11) 97777-6666', '2026-04-05 08:00:00'),
('Gabriela Nunes',  'gabi.nunes@empresa.com.br','(41) 3333-2222',  '2026-05-19 17:10:00'),
('Ana Souza',       'ana.souza@gmail.com',      '(11) 98765-4321', '2026-06-01 12:00:00');
```

(O último registro é propositalmente duplicado, para os exemplos de busca de duplicatas.)

🟢 Nível 1 — Básico

1. Listar todos os clientes

```
SELECT * FROM clientes;
```

2. Escolher colunas específicas

```
SELECT NOME, EMAIL FROM clientes;
```

3. Ordenar (mais recentes primeiro — igual à tela `clientes.php`)

```
SELECT ID, NOME, EMAIL, TELEFONE
FROM clientes
ORDER BY ID DESC;
```

4. Buscar um cliente pelo ID (usado em `atualizar.php`)

```
SELECT * FROM clientes WHERE ID = 3;
```

5. Limitar a quantidade de resultados

```
SELECT NOME, EMAIL FROM clientes ORDER BY NOME LIMIT 5;
```

6. Inserir um cliente (usado em `cadastro.php`)

```
INSERT INTO clientes (NOME, EMAIL, TELEFONE)
VALUES ('Helena Castro', 'helena@gmail.com', '(11) 91234-5678');
```

7. Atualizar um cliente (usado em `atualizar.php`)

```
UPDATE clientes
SET NOME = 'Helena Castro Silva', TELEFONE = '(11) 90000-1111'
WHERE ID = 9;
```

> ⚠ Nunca esqueça o `WHERE` em `UPDATE` / `DELETE` — sem ele, **todas** as linhas são alteradas.

8. Excluir um cliente (usado em `apagar.php`)

```
DELETE FROM clientes WHERE ID = 9;
```

🟡 Nível 2 — Filtros e funções

9. Busca por parte do nome (`LIKE`)

```
SELECT * FROM clientes WHERE NOME LIKE '%ana%';
```

10. Busca em várias colunas (equivalente à busca da tela de listagem)

```
SELECT ID, NOME, EMAIL, TELEFONE
FROM clientes
WHERE NOME     LIKE '%gmail%'
   OR EMAIL    LIKE '%gmail%'
   OR TELEFONE LIKE '%gmail%';
```

11. Clientes sem telefone (`IS NULL`)

```
SELECT NOME, EMAIL FROM clientes
WHERE TELEFONE IS NULL OR TELEFONE = '';
```

12. Clientes de uma lista de DDDs (`IN`)

```
SELECT NOME, TELEFONE
FROM clientes
WHERE SUBSTRING(TELEFONE, 2, 2) IN ('11', '21');
```

13. Cadastrados em um intervalo de datas (`BETWEEN`)

```
SELECT NOME, CRIADO_EM
FROM clientes
WHERE CRIADO_EM BETWEEN '2026-02-01' AND '2026-03-31 23:59:59';
```

14. Contar quantos clientes existem (`COUNT`)

```
SELECT COUNT(*) AS total_clientes FROM clientes;
```

15. Formatando a saída (funções de texto e data)

```
SELECT
    UPPER(NOME)                              AS nome_maiusculo,
    LOWER(EMAIL)                             AS email,
    COALESCE(TELEFONE, 'Não informado')      AS telefone,
    DATE_FORMAT(CRIADO_EM, '%d/%m/%Y %H:%i') AS cadastrado_em
FROM clientes;
```

16. Paginação (página 2, 3 registros por página)

```
SELECT ID, NOME, EMAIL
FROM clientes
ORDER BY ID
LIMIT 3 OFFSET 3;   -- OFFSET = (página - 1) * itens_por_página
```

🟠 Nível 3 — Agrupamentos e subconsultas

17. Quantidade de clientes por provedor de email (`GROUP BY`)

```
SELECT
    SUBSTRING_INDEX(EMAIL, '@', -1) AS dominio,
    COUNT(*)                         AS quantidade
FROM clientes
GROUP BY dominio
ORDER BY quantidade DESC;
```

18. Apenas domínios com mais de 1 cliente (`HAVING`)

```
SELECT SUBSTRING_INDEX(EMAIL, '@', -1) AS dominio, COUNT(*) AS quantidade
FROM clientes
GROUP BY dominio
HAVING COUNT(*) > 1;
```

19. Cadastros por mês

```
SELECT
    DATE_FORMAT(CRIADO_EM, '%Y-%m') AS mes,
    COUNT(*)                        AS novos_clientes
FROM clientes
GROUP BY mes
ORDER BY mes;
```

20. Classificar o tipo de telefone (`CASE`)

```
SELECT
    NOME,
    TELEFONE,
    CASE
        WHEN TELEFONE IS NULL OR TELEFONE = ''                    THEN 'Sem telefone'
        WHEN LENGTH(REGEXP_REPLACE(TELEFONE, '[^0-9]', '')) = 11 THEN 'Celular'
        WHEN LENGTH(REGEXP_REPLACE(TELEFONE, '[^0-9]', '')) = 10 THEN 'Fixo'
        ELSE 'Formato desconhecido'
    END AS tipo_telefone
FROM clientes;
```

21. Encontrar emails duplicados

```
SELECT EMAIL, COUNT(*) AS vezes, GROUP_CONCAT(ID ORDER BY ID) AS ids
FROM clientes
GROUP BY EMAIL
HAVING COUNT(*) > 1;
```

22. Subconsulta: o cliente cadastrado mais recentemente

```
SELECT NOME, EMAIL, CRIADO_EM
FROM clientes
WHERE CRIADO_EM = (SELECT MAX(CRIADO_EM) FROM clientes);
```

23. Subconsulta com `IN`: clientes dos domínios mais populares

```
SELECT NOME, EMAIL
FROM clientes
WHERE SUBSTRING_INDEX(EMAIL, '@', -1) IN (
    SELECT dominio FROM (
        SELECT SUBSTRING_INDEX(EMAIL, '@', -1) AS dominio
        FROM clientes
        GROUP BY dominio
        HAVING COUNT(*) >= 2
    ) AS populares
);
```

24. `JOIN` entre tabelas: clientes que também são usuários do sistema

```
SELECT c.ID AS id_cliente, c.NOME, c.EMAIL, u.ID AS id_usuario
FROM clientes c
INNER JOIN usuarios u ON u.EMAIL = c.EMAIL;
```

25. `LEFT JOIN`: clientes que não são usuários

```
SELECT c.NOME, c.EMAIL
FROM clientes c
LEFT JOIN usuarios u ON u.EMAIL = c.EMAIL
WHERE u.ID IS NULL;
```

26. `UNION`: lista única de todos os contatos (clientes + usuários)

```
SELECT NOME, EMAIL, 'Cliente' AS origem FROM clientes
UNION
SELECT NOME, EMAIL, 'Usuário' AS origem FROM usuarios
ORDER BY NOME;
```

🔴 Nível 4 — Avançado

> As consultas com `WITH` (CTE) e funções de janela (`OVER`) exigem **MySQL 8+** ou **MariaDB 10.2+** (as versões atuais do XAMPP atendem).

27. CTE (`WITH`): percentual de clientes por domínio

```
WITH por_dominio AS (
    SELECT SUBSTRING_INDEX(EMAIL, '@', -1) AS dominio, COUNT(*) AS qtd
    FROM clientes
    GROUP BY dominio
)
SELECT
    dominio,
    qtd,
    ROUND(qtd * 100.0 / (SELECT SUM(qtd) FROM por_dominio), 1) AS percentual
FROM por_dominio
ORDER BY qtd DESC;
```

28. Numerar os clientes dentro de cada domínio (`ROW_NUMBER`)

```
SELECT
    NOME,
    EMAIL,
    ROW_NUMBER() OVER (
        PARTITION BY SUBSTRING_INDEX(EMAIL, '@', -1)
        ORDER BY CRIADO_EM
    ) AS ordem_no_dominio
FROM clientes;
```

29. Total acumulado de cadastros ao longo do tempo (*running total*)

```
SELECT
    DATE_FORMAT(CRIADO_EM, '%Y-%m') AS mes,
    COUNT(*)                        AS novos,
    SUM(COUNT(*)) OVER (ORDER BY DATE_FORMAT(CRIADO_EM, '%Y-%m')) AS acumulado
FROM clientes
GROUP BY mes
ORDER BY mes;
```

30. Intervalo entre um cadastro e o anterior (`LAG`)

```
SELECT
    NOME,
    CRIADO_EM,
    LAG(CRIADO_EM) OVER (ORDER BY CRIADO_EM) AS cadastro_anterior,
    DATEDIFF(CRIADO_EM, LAG(CRIADO_EM) OVER (ORDER BY CRIADO_EM)) AS dias_desde_anterior
FROM clientes;
```

31. Remover duplicatas mantendo o registro mais antigo

```
-- 1) Conferir o que será apagado
SELECT c1.*
FROM clientes c1
JOIN clientes c2 ON c1.EMAIL = c2.EMAIL AND c1.ID > c2.ID;

-- 2) Apagar
DELETE c1
FROM clientes c1
JOIN clientes c2 ON c1.EMAIL = c2.EMAIL AND c1.ID > c2.ID;
```

32. Impedir emails duplicados daqui para frente (índice único)

```
ALTER TABLE clientes ADD UNIQUE KEY UQ_CLIENTES_EMAIL (EMAIL);
```

33. Índice para acelerar buscas por nome + análise com `EXPLAIN`

```
CREATE INDEX IDX_CLIENTES_NOME ON clientes (NOME);

EXPLAIN SELECT * FROM clientes WHERE NOME LIKE 'Ana%';
-- Observação: LIKE '%ana%' (curinga no início) NÃO consegue usar o índice.
```

34. Normalizar telefones: deixar só os dígitos

```
SELECT NOME, TELEFONE, REGEXP_REPLACE(TELEFONE, '[^0-9]', '') AS somente_digitos
FROM clientes
WHERE TELEFONE IS NOT NULL;
```

35. *Upsert*: inserir ou atualizar se o email já existir *(requer o índice único do item 32)*

```
INSERT INTO clientes (NOME, EMAIL, TELEFONE)
VALUES ('Ana Souza', 'ana.souza@gmail.com', '(11) 95555-4444')
ON DUPLICATE KEY UPDATE
    NOME     = VALUES(NOME),
    TELEFONE = VALUES(TELEFONE);
```

36. View: relatório pronto para consulta

```
CREATE OR REPLACE VIEW vw_clientes_resumo AS
SELECT
    ID,
    NOME,
    EMAIL,
    SUBSTRING_INDEX(EMAIL, '@', -1)          AS dominio,
    COALESCE(TELEFONE, 'Não informado')      AS telefone,
    DATE_FORMAT(CRIADO_EM, '%d/%m/%Y')       AS data_cadastro
FROM clientes;

-- Uso:
SELECT * FROM vw_clientes_resumo WHERE dominio = 'gmail.com';
```

37. Transação: várias operações "tudo ou nada"

```
START TRANSACTION;

UPDATE clientes SET TELEFONE = NULL WHERE TELEFONE = '';
DELETE FROM clientes WHERE EMAIL NOT LIKE '%@%.%';

-- Conferiu e está tudo certo?
COMMIT;
-- Algo deu errado? Desfaça com:
-- ROLLBACK;
```

38. Stored procedure: busca reutilizável

```
DELIMITER //

CREATE PROCEDURE sp_buscar_clientes(IN termo VARCHAR(100))
BEGIN
    SELECT ID, NOME, EMAIL, TELEFONE
    FROM clientes
    WHERE NOME     LIKE CONCAT('%', termo, '%')
       OR EMAIL    LIKE CONCAT('%', termo, '%')
       OR TELEFONE LIKE CONCAT('%', termo, '%')
    ORDER BY NOME;
END //

DELIMITER ;

-- Uso:
CALL sp_buscar_clientes('gmail');
```

39. Trigger + tabela de auditoria: registrar toda exclusão

```
CREATE TABLE IF NOT EXISTS clientes_log (
    ID          INT AUTO_INCREMENT PRIMARY KEY,
    CLIENTE_ID  INT,
    NOME        VARCHAR(100),
    EMAIL       VARCHAR(100),
    ACAO        VARCHAR(10),
    DATA_ACAO   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER trg_clientes_delete
AFTER DELETE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO clientes_log (CLIENTE_ID, NOME, EMAIL, ACAO)
    VALUES (OLD.ID, OLD.NOME, OLD.EMAIL, 'DELETE');
END //

DELIMITER ;

-- Depois de apagar um cliente pelo sistema, confira:
SELECT * FROM clientes_log ORDER BY DATA_ACAO DESC;
```

40. Painel de indicadores em uma única consulta

```
SELECT
    (SELECT COUNT(*) FROM clientes)                                    AS
total_clientes,
    (SELECT COUNT(*) FROM clientes WHERE TELEFONE IS NULL OR TELEFONE = '') AS
sem_telefone,
    (SELECT COUNT(DISTINCT SUBSTRING_INDEX(EMAIL, '@', -1)) FROM clientes) AS
dominios_distintos,
    (SELECT COUNT(*) FROM clientes
       WHERE CRIADO_EM >= DATE_SUB(NOW(), INTERVAL 30 DAY))            AS
novos_ultimos_30_dias,
    (SELECT COUNT(*) FROM usuarios)                                    AS
usuarios_do_sistema;
```

🔐 Consultas úteis para a tabela `usuarios`

Listar usuários sem expor o hash da senha

```
SELECT ID, NOME, EMAIL, TELEFONE, CRIADO_EM FROM usuarios;
```

Consulta feita pelo login (`login.php`)

```
SELECT ID, NOME, EMAIL, SENHA FROM usuarios WHERE EMAIL = 'admin@sistema.com';
```

Trocar a senha de um usuário

Gere o hash em PHP (nunca grave a senha em texto puro):

```
php -r "echo password_hash('NovaSenha@123', PASSWORD_DEFAULT);"
```

e atualize no banco:

```
UPDATE usuarios
SET SENHA = '$2y$10$...hash_gerado_acima...'
WHERE EMAIL = 'admin@sistema.com';
```

Projeto educacional — uso livre.